الإصدار 1.0

Spring Security

Spring Security

مقدمة: Spring Security

إطار عمل مسؤول عن حماية تطبيقات Spring Boot. هدفه الإجابة عن سؤالين.

من هو المستخدم؟

Authentication

ماذا يُسمح له أن يفعل؟

Authorization

مجالات الحماية

- Session Management
- Password Encryption
- API Protection
- CSRF Protection

Spring Security

Authentication & Authorization

AUTHENTICATION · المصادقة

من أنت؟

التحقق من الهوية باستخدام username وpassword.

Authentication Success | Authentication Failed

AUTHORIZATION · التفويض

ما الذي يمكنك فعله؟

تحديد الصلاحيات بعد نجاح الدخول.

ADMIN يمتلك جميع العمليات.

USER يمتلك عمليات محددة.

Spring Security

Security Architecture: كل طبقة لها مهمة واحدة

تطبيق مبدأ Separation of Concerns.

1

Client

2

Spring Security

3

AuthenticationProvider

4

UserDetailsService

5

Database

6

SecurityContext

7

Controller

نقطة إعداد الحماية: SecurityConfig

الكلاس المسؤول عن جميع إعدادات الحماية.

```
@Configuration @EnableWebSecurity public class SecurityConfig {}
```

@Configuration

يحتوي Beans يديرها Spring.

@EnableWebSecurity

تفعيل Spring Security داخل التطبيق.

UserDetailsService و UserDetails

يتعامل Spring Security مع الواجهة UserDetails وليس مع أي User مباشرة.

UserDetails

مثل MyUser implements UserDetails. الحقول هي id، username، password، role.

MyUser implements UserDetails

UserDetailsService

يخبر Spring كيف يجلب المستخدم من قاعدة البيانات عبر loadUserByUsername().

loadUserByUsername()

نتيجة البحث

إن وُجد المستخدم، يُرجع المستخدم. وإن لم يُوجد، تكون النتيجة UsernameNotFoundException.

Spring Security

1 جلب المستخدم

2 مقارنة كلمة المرور

AuthenticationProvider: من يتحقق من الدخول

DaoAuthenticationProvider

نستخدمه لأن المستخدمين مخزنون في قاعدة بيانات.

DAO = Data Access Object

3 إنشاء Authentication Object

4 قبول أو رفض الدخول

Password Encoding: لا تخزّن كلمة المرور نصًا

المشكلة

123456

تخزين كلمة المرور كما هي.

الحل

BCryptPasswordEncoder

```
$2a$10$sd87Hsh...
```

الفائدة

- حماية كلمات المرور.
- تقليل مخاطر الاختراق.
- الالتزام بالممارسات الأمنية.

AuthenticationPrincipal: المستخدم الحالي بأمان

نستخدم

`myUser.getId()`

بدلًا من

POST /todos?userId=5

كود: addTodo(@AuthenticationPrincipal MyUser myUser, @RequestBody Todo todo)

الميزة الأمنية

منع التلاعب بمعرف المستخدم.

Authority مقابل Role

من أهم المفاهيم في Spring Security. Role واحد يمتلك عدة Authorities.

Role	Authority
دور وظيفي High Level	صلاحية محددة Fine Grained
ROLE_ADMIN ROLE_USER ROLE_MANAGER	CREATE_USER DELETE_USER VIEW_REPORTS APPROVE_REQUEST READ_PROFILE

الدور والصلاحيات التابعة له

ROLE_ADMIN

CREATE_USER، DELETE_USER، UPDATE_USER، READ_USER

ROLE_USER

READ_USER

Spring Security

Security Flow: رحلة تسجيل الدخول

1 User Login

2 Spring Security

3 UserDetailsService

4 Database

5 UserDetails

6 AuthenticationProvider

7 Password Validation

8 Security Context

9 Controller Access

SecurityConfig كامل

الكود الكامل مع DaoAuthenticationProvider، Beans: PasswordEncoder (BCrypt)، SecurityFilterChain، AuthenticationManager.

@Configuration · @EnableWebSecurity · csrf.disable()

- /auth/** مفتوح للجميع (permitAll)
- /admin/** لدور ADMIN فقط
- /user/** لدور USER أو ADMIN
- أي طلب آخر يتطلب authenticated
- csrf مُعطّل في هذا المثال

اختبار SecurityConfig

أضف الاعتمادية spring-security-test بنطاق test، ثم اختبر أربع حالات.

Endpoint عام (/public)

```
mockMvc.perform(get("/public"))
    .andExpect(status().isOk());
```

مستخدم مجهول على /user

```
mockMvc.perform(get("/user"))
    .andExpect(status().isUnauthorized());
```

@WithMockUser ADMIN على /admin

```
@WithMockUser(roles = "ADMIN")
// داخل الاختبار
mockMvc.perform(get("/admin"))
    .andExpect(status().isOk());
```

@WithMockUser USER على /admin

```
@WithMockUser(roles = "USER")
// داخل الاختبار
mockMvc.perform(get("/admin"))
    .andExpect(status().isForbidden());
```

Spring Security

اختبار CSRF

POST بدون Token

Forbidden

```
mockMvc.perform(post("/user"))
;andExpect(status().isForbidden()).
```

POST مع with(csrf())

OK

```
mockMvc.perform(post("/user").with(csrf()))
;andExpect(status().isOk()).
```

Token غير صالح

Forbidden

```
mockMvc.perform(post("/user")
(with(csrf().useInvalidToken()).
;andExpect(status().isForbidden()).
```

Spring Security

Common Mistakes و Best Practices

لا تفعل

- تخزين كلمات المرور نصًا.
- إعطاء الجميع ROLE_ADMIN.
- تعطيل الحماية أثناء التطوير ونسيانها.
- استخدام UserId من الطلب.
- ترك Authorization وCSRF بلا اختبار.

افعل

- BCrypt دائمًا.
- Roles واضحة.
- AuthenticationPrincipal.
- فصل Security عن Business Logic.
- Security Tests.
- Principle of Least Privilege.

Interview Questions

ما الفرق بين Authentication وAuthorization؟

Authentication: من أنت؟ Authorization: ماذا تستطيع أن تفعل؟

ما مهمة UserDetailsService؟

جلب المستخدم من قاعدة البيانات.

ما مهمة AuthenticationProvider؟

تنفيذ المصادقة والتحقق.

لماذا BCrypt؟

لتخزين كلمات المرور بأمان.

ما الفرق بين Role وAuthority؟

Role هو Business Role، وAuthority هي Permission.

ما وظيفة @AuthenticationPrincipal؟

المستخدم الحالي من Security Context.

Spring Security

الخلاصة والخطوات التالية

البنية القياسية في معظم مشاريع Spring Boot الاحترافية.

UserDetails
↓
UserDetailsService
↓
AuthenticationProvider
↓
PasswordEncoder
↓
SecurityContext
↓
Controller

انتهى